Этап 1. Сбор и анализ требований

Изучение предметной области

1. Что это такое

Предметная область — это фрагмент реального мира, который автоматизируется. Для БД это совокупность объектов, процессов и правил, которые нужно отразить в данных. До проектирования схемы нужно понять, **как всё устроено на самом деле**, а не как кажется.

2. Зачем это нужно

Ошибки на этом этапе — самые дорогие. Если неверно выделить процессы или пропустить ограничение, то на этапе реализации выяснится, что схема не отражает реальность. Переделка модели данных стоит на порядок дороже, чем уточнение требования на старте.

3. Методы изучения

Метод	Что даёт	Когда применять
Интервью с заказчиком	Понимание целей, приоритетов, неочевидных правил	Всегда — на старте
Анализ документов	Существующие бланки, формы, отчёты, регламенты	Если есть бумажный или устаревший учёт
Наблюдение за работой	Реальные сценарии, которые заказчик не озвучит	Если процесс ручной и можно посмотреть
Анализ аналогов	Понимание типовых решений, чего не хватает	Если есть похожие системы на рынке

4. Что именно нужно получить

На выходе — **список ключевых процессов** и **список ограничений**.

1) **Процесс** — это цепочка действий, которая имеет бизнес-смысл. Например:

- регистрация нового читателя;
- выдача книги;
- возврат книги;
- списание экземпляра;
- инвентаризация фонда.

2) **Ограничение** — это правило, которое нельзя нарушить. Ограничения бывают трёх типов:

Тип	Пример	Как реализуется
Структурные	У каждой книги есть хотя бы один автор	NOT NULL, FK
Бизнес-правила	Читатель может держать не более 5 книг	CHECK, триггер, логика приложения
Технические	ISBN уникален в пределах БД	UNIQUE, PRIMARY KEY

Как выделить процессы

Последовательность:

1. Определить, **кто** участвует (роли: читатель, библиотекарь, администратор).
2. Определить, **что** они делают (действия: берёт книгу, оформляет выдачу).
3. Определить, **в каком порядке** (читатель приходит → ищет книгу → библиотекарь оформляет выдачу → книга уходит).
4. Определить, **что меняется в данных** на каждом шаге (статус книги: «на полке» → «выдана», появляется запись о выдаче).

Как выделить ограничения

Источники ограничений:

- **Закон и регламент** (срок хранения фонда, правила списания).
- **Логика процесса** (нельзя вернуть то, что не выдано).

- **Физическая реальность** (один экземпляр — у одного читателя).
- **Бизнес-политика** (штраф за просрочку, лимит книг на руки).

Типичные ошибки

- **«Заказчик сам всё знает»** — заказчик описывает идеальный процесс, а в реальности есть исключения и обходные пути. Нужно уточнять граничные случаи.
- **«Это очевидно»** — правило кажется самоочевидным, но не зафиксировано. Потом спор о трактовке.
- **«Добавим потом»** — ограничение пропущено, и на этапе реализации выясняется, что схема его не поддерживает.
- **«Слишком детально»** — на этом этапе не нужно описывать каждую колонку, достаточно понимать суть процесса и ограничения.

Роли пользователей

1. Что это такое

Роль — это набор прав и действий, доступных определённому типу пользователя. Роль не равна конкретному человеку: один человек может иметь несколько ролей (библиотекарь и читатель одновременно).

2. Зачем нужны роли на этапе анализа

Роли определяют:

- **какие данные** доступны (читатель видит свои выдачи, библиотекарь — все);
- **какие действия** разрешены (читатель не может списать книгу);
- **какие запросы** будут частыми (поиск книг — для всех, отчёты — для администратора).

3. Как выделять роли

1) Подход «сверху вниз»:
   a) Выписать все действия, которые возможны в системе.
   b) Сгруппировать их по типам пользователей.
   c) Проверить: нет ли действий без роли (забыли) и нет ли ролей без действий (лишние).
2) Подход «снизу вверх»:
   a) Назвать роли, которые очевидны из предметной области.
   b) Для каждой роли — что она делает в реальности.
   c) Проверить: охвачены ли все процессы.

4. Пример для библиотеки

Роль	Действия
Читатель	Регистрируется, ищет книги, смотрит свои выдачи, продлевает срок, видит штрафы
Библиотекарь	Оформляет выдачу, принимает возврат, начисляет штраф, размещает книги по залам, ищет книгу для читателя
Администратор	Управляет залами и стеллажами, категориями книг, формирует отчёты, списывает экземпляры, управляет пользователями

5. Что важно зафиксировать

Для каждой роли — не просто список действий, а:

- **входные данные** (что нужно для выполнения действия);
- **результат** (что меняется в данных);
- **ограничения** (что нельзя делать).

Бизнес-правила

1. Что это такое

Бизнес-правило — это утверждение о том, что должно быть истинно в данных всегда. Например: «сумма штрафа не может быть отрицательной» или «нельзя выдать книгу, которая уже на руках».

2. Чем бизнес-правило отличается от ограничения СУБД

Бизнес-правило	Ограничение СУБД
Формулируется на языке предметной области	Формулируется на языке БД (CHECK, FK, UNIQUE)
«Читатель не может держать более 5 книг»	Триггер, который считает активные выдачи и отклоняет новую
«Срок выдачи — 14 дней для обычных книг»	Логика приложения при создании выдачи

Не каждое бизнес-правило можно выразить одним ограничением СУБД. Некоторые требуют логики приложения или триггеров. Но на этапе анализа важно их все зафиксировать.

3. Классификация бизнес-правил

Категория	Пример	Где реализуется
Ограничение значения	Штраф ≥ 0	CHECK в таблице
Ограничение уникальности	ISBN уникален	UNIQUE
Ограничение ссылки	Каждая выдача ссылается на конкретный экземпляр	FOREIGN KEY
Ограничение состояния	Книга в статусе «выдана» не может быть выдана снова	Логика приложения + частичный UNIQUE-индекс
Ограничение количества	Не более 5 книг на руках	Триггер или логика приложения
Ограничение по времени	Дата возврата ≥ дата выдачи	CHECK

4. Как формулировать

Шаблон: **«[условие] должно/не должно выполняться»**.

Плохо	Хорошо
«книги не просрочены» (непонятно, что это значит)	фактическая дата возврата не может быть раньше даты выдачи
мало книг у читателя» (количественно не определено)	у одного читателя не может быть более 5 активных выдач одновременно

5. Сколько правил нужно

Для учебного проекта — 5–7. В реальных проектах — десятки.

Критерий: правила должны покрывать все ключевые процессы и все ограничения, нарушение которых приведёт к некорректным данным.

Состав данных и объёмы

1. Состав данных

Для каждого действия, определённого на шаге 2, нужно выписать:

- **какие данные нужны на входе** (для оформления выдачи — читатель, экземпляр, дата);
- **какие данные создаются или изменяются** (новая запись в «Выдача», статус экземпляра меняется на «выдан»);
- **какие данные нужны для отображения** (ФИО читателя, название книги, даты).

Результат — **список атрибутов по сущностям**. На этом этапе типы данных абстрактные (строка, число, дата), без привязки к СУБД.

2. Объёмы данных

Оценка объёмов нужна для:

- выбора типов данных (для большой таблицы — BIGINT вместо INT);
- решения об индексах (таблица на 10 млн строк без индекса — это медленно);
- решения о партиционировании (если таблица растёт на миллионы строк в год — партиции по году);
- оценки стоимости хранения и производительности.

3. Как оценивать

Метод: от количества операций к количеству строк.

Вопрос	Пример для библиотеки
Сколько объектов в реальности?	~10 000 книг, ~3 000 читателей
Сколько операций в день?	~100 выдач/возвратов в день
Сколько в год?	~36 000 операций в год → ~36 000 записей в «Выдача» за год
Рост?	+10% читателей в год

4. Что оценивать

Таблица	Оценка на 1 год	Оценка на 5 лет
Книга (экземпляры)	10 000	12 000
Читатель	3 000	5 000
Выдача	36 000	180 000
Штраф	3 000	15 000

Оценки не обязаны быть точными — они должны показать порядок величины.

Частота операций

1. Зачем это нужно

Частота операций определяет архитектуру:

- **частое чтение** → нужны индексы на поля поиска, возможно кэширование;
- **частая запись** → минимум ограничений на путь записи, простые INSERT;
- **редкие тяжёлые отчёты** → можно не оптимизировать, выполнять ночью, или вынести в отдельную БД (OLAP);
- **массовые обновления** → batching, нерабочее время.

2. Виды операций

Тип	Описание	Пример
OLTP (транзакционная)	Много коротких операций: чтение, вставка, обновление одной записи	Оформление выдачи, поиск книги
OLAP (аналитическая)	Мало операций, но каждая обрабатывает много данных	Отчёт по загрузке залов за квартал
Пакетная	Массовая обработка данных, обычно по расписанию	Инвентаризация, начисление штрафов за просрочку

3. Как анализировать

Для каждого процесса из пункта 1 указать:

- **тип операции** (чтение, запись, обновление, удаление);
- **частоту** (в секунду, в минуту, в день, в месяц);
- **важность** (критичная для пользователя, фоновая).

Пример таблицы

Процесс	Тип	Частота	Важность
Поиск книги	Чтение	500/день	Высокая
Оформление выдачи	Запись	100/день	Высокая
Возврат книги	Обновление	100/день	Высокая
Начисление штрафов	Пакетная	1/день	Средняя
Отчёт по загрузке залов	Чтение (OLAP)	1/неделю	Средняя
Инвентаризация	Пакетная	1/год	Низкая

Требования к производительности

1. Что это такое

Требование к производительности — это ожидаемое время ответа системы на конкретную операцию при заданной нагрузке.

2. Зачем это нужно на этапе анализа

Производительность зависит от архитектуры БД: индексов, партиционирования, типов данных. Если известно, что поиск свободных книг должен работать за 100 мс, а в таблице 10 млн строк, — индекс на поле поиска обязателен, и это решение нужно принять на этапе физического проектирования, а не после релиза.

3. Виды требований

Тип	Пример
Время ответа	Поиск книги — не более 200 мс
Пропускная способность	Оформление выдачи — не менее 10 операций в минуту
Доступность	Система доступна 24/7, плановый простой — не более 1 часа в месяц
Масштабируемость	При росте до 50 000 читателей — время ответа не должно вырасти более чем в 2 раза

4. Как формулировать

Шаблон: **«[операция] должна выполняться не более [время] при [нагрузка]»**.

Плохо	Хорошо
система должна быть быстрой	поиск свободных книг по жанру и залу — не более 200 мс при 100 одновременных запросах
отчёты не должны тормозить	отчёт по загрузке залов за квартал — не более 5 секунд при объёме 100 000 записей о выдачах

5. Что важно учесть

- **Не все запросы одинаково важны.** Поиск книги — критичен (читатель ждёт). Отчёт за квартал — может подождать 5 секунд.
- **Реалистичные цифры.** Не нужно требовать 10 мс на сложный JOIN по 3 таблицам без индексов. Но 200 мс на простой поиск с индексом — разумно.
- **Пиковая нагрузка.** Если в обеденный перерос 50 человек одновременно ищут книги — это пик, и система должна его выдержать.

Итог: что должен содержать результат этапа 1

Пункт в документе	Содержание
Описание процессов	Ключевые процессы и их шаги
Роли пользователей	Роли, действия, права
Бизнес-правила	5–7 формализованных правил
Состав данных	Список атрибутов по сущностям с типами (абстрактно)
Оценка объёмов	Количество записей в основных таблицах на 1 и 5 лет
Частота операций	Таблица: процесс → тип → частота → важность
Требования к производительности	3–5 конкретных требований с метриками
Список сущностей и их атрибутов	Таблица с колонками: Сущность → Атрибут → Тип данных → Обязательность (да/нет) → Уникальность (да/нет)

Этот набор — основа для концептуального проектирования (этап 2): сущности и связи строятся именно из процессов и бизнес-правил, а индексы и типы данных (этап 4) — из требований к производительности и оценок объёмов.